# Talking to GemStone/S from Rust

A complete article-style guide to gemstone-rs

Generated from the Markdown sources in gemstone-rs/docs.

# Contents

# Talking to GemStone/S from Rust - A Complete Guide to gemstone-rs

*GemStone/S stores live objects. Rust gives you fast, explicit systems code.* `gemstone-rs` *connects the two without putting Python in the process.*

---

# What is GemStone/S?

GemStone/S 64 is an object database and application server for Smalltalk systems. It stores objects directly, supports many concurrent sessions, and lets clients send Smalltalk messages to live objects. Instead of translating your application into rows and joins, you talk to objects in the stone.

Rust is a good fit for a direct GemStone client because Rust applications often need predictable resource management, explicit error handling, and clean deployment. `gemstone-rs` is the Rust bridge for that job.

---

# The Architecture

```
Rust application
      |
      v
gemstone-rs             safe Rust API
      |
      v
gemstone-gci            dynamic libgcirpc loader and raw ABI calls
      |
      v
GCI C library           ships with GemStone/S
      |
      v
GemStone stone
```

The split matters:

| Layer | Responsibility |
|---|---|
| `gemstone-gci` | Load `libgcirpc`, expose raw GCI calls, keep unsafe ABI code isolated. |
| `gemstone-rs` | Provide `Config`, `Session`, `Oop`, `Value`, transactions, browser API, and codegen. |
| `gemstone-rs-cli` | Provide `gemstone-rs eval`, browse, inspect, and codegen commands. |
| `gemstone-rs-explorer` | Provide a local-only HTTP explorer for browsing and codegen workflows. |
| `gemstone-rs Workbench` | Add VS Code sidebar browsing and codegen actions over the CLI. |

This gives Rust users a native path to GemStone/S, while Python remains only one possible consumer through `gemstone-py`.

---

# Install

For Rust applications:

```
cargo add gemstone-rs
```

For command-line tools:

```
cargo install gemstone-rs-cli
cargo install gemstone-rs-explorer
```

For VS Code:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Runtime configuration comes from the same environment variables used by the CLI, explorer, and workbench:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is canonical. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent. Set `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

---

# First Login

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let value = session.eval("3 + 4")?;
    assert_eq!(value, Value::SmallInt(7));
```

```
    session.logout()?;
    Ok(())
}
```

Run the same check from the CLI:

```
gemstone-rs env sample
gemstone-rs env write
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --json
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check gemstone-rs.codegen
gemstone-rs-explorer --env-file .env.gemstone-rs --port 8787
```

`env sample` prints a copy-pasteable setup script with placeholders for passwords, so a new shell can be configured without accidentally dumping secrets into docs, tickets, or chat. `env write` saves the same template to `.env.gemstone-rs` and refuses to overwrite unless `--force` is used. `--env-file` is now a global CLI option, so doctor, eval, browse, inspect, BridgeRoot, and codegen commands can all use the same file without requiring you to source it globally. The explorer can also start with `--env-file .env.gemstone-rs`.

The doctor report names the source used to select `libgcirpc`: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. Failed checks now include remediation hints, so CLI, VS Code, and CI diagnostics are easier to compare and act on. `doctor --strict` makes CI fail when the stone or GCI library source is only coming from defaults, and GCI diagnostics show whether the selected `libgcirpc` exists, is a file, is readable, and appears to match arm64 or x86_64.

Expected output:

```
7
```

---

# OOPs and Values

GemStone objects are identified by OOPs. `gemstone-rs` keeps that explicit:

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
```

```
let seven = session.value_to_oop(&Value::SmallInt(7))?;
let printed = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed)?);
```

`Value` covers the simple automatic conversions:

| GemStone result | Rust value |
|---|---|
| `nil` | `Value::Nil` |
| `true` / `false` | `Value::Bool` |
| `SmallInteger` | `Value::SmallInt` |
| `Character` | `Value::Char` |
| fetched string values | `Value::String` |
| other live objects | `Value::Oop` |

For long-lived raw OOPs, retain them in the GemStone export set:

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases on drop if you do not call `release()`.

---

# Transactions

GemStone sessions are transactional. `gemstone-rs` gives you manual primitives and a small transaction wrapper:

```
session.transaction(|session| {
    let value = session.new_string("committed from Rust")?;
    session.global_put("GemStoneRsExample", value)
})?;
```

If the closure returns `Ok`, the transaction commits. If it returns `Err`, the session aborts.

Manual control is also available:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
session.commit()?;
session.abort()?;
```

---

# Browser API

The reusable browser API is the foundation for the CLI, explorer, and VS Code sidebar:

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);

let dictionaries = browser.dictionaries()?;
let classes = browser.classes("UserGlobals")?;
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

CLI equivalents:

```
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
```

# Codegen

Codegen turns a small, reviewable config into checked-in Rust wrappers.

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Commands:

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs codegen diff examples/codegen/gemstone-rs.codegen
```

```
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain-profile --json default examples/codegen/gemstone-
rs.codegen-profiles.json
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

Generated wrapper files now include `#[cfg(test)]` stubs for surface names, method metadata, and mapped-field metadata. `codegen explain` reports those stubs beside the classes, selectors, argument counts, return helpers, field keys, key policies, and mapped fields that will be generated. Add `--json` for the explorer or VS Code when they need the same summary as structured data. `codegen explain-profile` gives the same report after resolving a named project profile, which is useful when the committed profile file is the source of truth for generation. The repository also commits schemas for the codegen model, project profiles, and `codegen explain --json` output, so editor panels and release checks can reason about the same structures. The JSON summary now includes each argument's config type and generated Rust type, so the explorer and VS Code can show the conversion before a file is written.

The newest wrapper polish is typed method arguments. An argument can remain an explicit `Oop`, or the config can ask codegen to accept native Rust values and convert them at the call boundary:

```
method = UserGlobals:OkzBooking class>>findById: | args=id:SmallInt | return=Oop
method = Object>>perform: | args=selector:Symbol
method = UserGlobals:Order>>statusSymbol | return=Symbol
method = UserGlobals:User>>named:active: | args=name:String,active:Bool | return=Oop
```

That produces wrapper methods shaped for normal Rust callers:

```
pub fn find_by_id(&mut self, id: i64) -> Result<Oop> {
    let id = self.session.smallint_oop(id);
    let value = self.session.perform(self.oop, "findById:", &[id])?;
    // typed return conversion follows
}

pub fn perform(&mut self, selector: impl AsRef<str>) -> Result<Value> {
    let selector = self.session.new_symbol(selector.as_ref())?;
    self.session.perform(self.oop, "perform:", &[selector])
}
```

Typed returns can now also use `Symbol`. The generated wrapper returns a Rust `String`, using the same explicit fetch path as `return=String`, but the config keeps the domain intent clear.

That is a direct catch-up item against `gemstone-py`: Python callers naturally pass Python ints and strings, while Rust now gets generated signatures that make those conversions explicit and compile-checked.

Generate a starter config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
```

Discovery now carries more useful metadata into that starter file. Source headers become stable Rust argument names, keyword selectors are the fallback, protocol names and the first source line become `doc=...` context, and types stay conservative until the developer narrows them. That is the right tradeoff for Rust: improve the wrapper shape from live GemStone metadata, but avoid unsafe guesses about argument and return types.

Generated wrappers keep selector spelling in one place:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
            expected: "String",
            actual: format!("{other:?}"),
        }),
    }
}
```

The output is normal Rust code. Check it in so reviews and editor indexing stay predictable.

---

# Object Mapping with BridgeRoot

Direct OOP access is important, but application code often wants normal Rust structs at the boundary. `gemstone-rs` now has a BridgeRoot mapping layer for that.

The default BridgeRoot is a GemStone `Dictionary` stored in `UserGlobals` under `#GemStoneRsBridgeRoot`. Rust can put typed payloads there, commit them, and read them back without hiding the lower-level OOP API.

The practical rule is simple: use `Oop` for low-level calls, `BridgeValue` for inspection, `BridgeMapped` for stable dictionary-backed payloads, `BridgeRoot` for a durable Rust-owned root, and `Remote<T>` when object identity matters. Codegen then turns the mapping into checked-in Rust source that can be reviewed and diffed like any other API boundary.

| Layer | Best for |
|---|---|
| `Oop` | raw object identity and direct selector sends |
| `BridgeValue` | exploratory reads, explorer UI, shape reports |

| Layer | Best for |
| --- | --- |
| `BridgeMapped` | typed Rust structs stored as GemStone dictionaries |
| `BridgeRoot` | stable root dictionary under `UserGlobals` |
| `Remote<T>` / `ObjectRef<T>` | explicit cached proxy over an existing OOP |
| codegen wrappers | committed selector and mapping APIs |

This is the deliberate difference from a fully transparent object model: normal Rust field access never performs network I/O, and dropping a Rust value never saves it back to GemStone. Reads use `refresh(&mut session)`, writes use `save(&mut session)` or an explicit transaction.

The smallest version stores a plain bridge value:

```
use gemstone_rs::{BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
]);

bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

For typed Rust code, derive `BridgeMapped`:

```
use gemstone_rs::{BridgeMapped, Config, Session};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    #[bridge(key_type = "Symbol")]
    name: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
    customer: CustomerDraft,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}
```

```
let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let draft = BookingDraft {
    amount: 100,
    customer: CustomerDraft {
        name: "Tariq".to_string(),
    },
    tags: vec!["priority".to_string(), "demo".to_string()],
    labels: BTreeMap::from([("source".to_string(), "article".to_string())]),
    note: None,
};

bridge_root.transaction(|root| {
    root.put_mapped("DerivedBookingDraft", &draft)?;
    let loaded: BookingDraft = root.get_mapped("DerivedBookingDraft")?;
    assert_eq!(loaded, draft);
    Ok(())
})?;
```

That example shows the important mapping choices:

| Feature | Why it matters |
|---|---|
| `#[derive(BridgeMapped)]` | Normal Rust structs can become BridgeRoot payloads. |
| `#[bridge(key = "...")]` | Rust field names do not have to match GemStone dictionary keys. |
| `key_type = "Symbol"` | Symbol-keyed dictionaries are explicit instead of accidental. |
| nested structs | nested dictionaries can read back into nested Rust structs. |
| `Vec<T>` | GemStone arrays can read back into Rust vectors. |
| `BTreeMap<String, T>` | string-keyed dictionaries can read back into Rust maps. |
| `Option<T>` | missing keys and GemStone `nil` can read back as `None`. |
| mapping errors | nested failures report paths like `booking.customer.name`, `tags[2]`, and `labels["source"]`. |
| `transaction` | BridgeRoot writes can commit on success and abort on error. |

Codegen can create the same mapping structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.tags | type=Vec<String> | key=tags
```

```
field = BookingDraft.labels | type=BTreeMap<String, String> | key=labels
field = BookingDraft.note | type=Option<String> | key=note
```

Map fields are intentionally string-keyed. Use `BTreeMap<String, T>` when the GemStone dictionary represents metadata or lookup values, and use a nested `BridgeMapped` struct when the related value has a stable domain shape. Codegen also accepts `Map<String, T>` as a shorter spelling and `Dictionary<T>` as an alias for `BTreeMap<String, T>`.

There is one important distinction: the key used to store a value in BridgeRoot is not the same as the keys inside a stored GemStone dictionary. `BTreeMap<String, T>` creates a string-keyed dictionary value:

```
use std::collections::BTreeMap;

let labels = BTreeMap::from([
    ("channel".to_string(), "web".to_string()),
    ("priority".to_string(), "high".to_string()),
]);

bridge_root.put_map("BookingLabels", &labels)?;
let labels: BTreeMap<String, String> = bridge_root.get_map("BookingLabels")?;
assert_eq!(labels["channel"], "web");
```

When Smalltalk code expects symbol keys inside the dictionary, build that dictionary explicitly:

```
use gemstone_rs::{BridgeKey, BridgeKeyType, BridgeValue};

let payload = BridgeValue::keyed_dictionary([
    (BridgeKey::symbol("status"), BridgeValue::from("ready")),
    (BridgeKey::symbol("amount"), BridgeValue::from(100_i64)),
    (BridgeKey::string("externalId"), BridgeValue::from("web-42")),
]);

bridge_root.put("SmalltalkBooking", payload)?;

let mut dictionary = bridge_root.get_dictionary("SmalltalkBooking")?;
assert_eq!(
    dictionary.at_string_with_key_type("status", BridgeKeyType::Symbol)?,
    "ready"
);
assert_eq!(
    dictionary.at_smallint_with_key_type("amount", BridgeKeyType::Symbol)?,
    100
);
assert_eq!(dictionary.at_string("externalId")?, "web-42");
```

Dynamic read-back preserves that distinction. A dictionary with only string keys comes back as `BridgeValue::Dictionary`; a dictionary with any symbol key comes back as `BridgeValue::KeyedDictionary`, so the explorer and VS Code webview can show the real Smalltalk-facing shape.

The next layer is an explicit remote object handle:

```
use gemstone_rs::{BridgeMapped, MaterializationProfile, Remote};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    status: String,
    amount: i64,
    labels: BTreeMap<String, String>,
}

let oop = bridge_root.get_oop("BookingDraft")?;
let mut remote = Remote::<BookingDraft>::with_type(oop, "UserGlobals:BookingDraft")
    .with_profile(MaterializationProfile::deep(4));

let mut loaded = remote.refresh(&mut session)?.clone();
loaded.status = "confirmed".to_string();
remote.set_value(loaded);
remote.save(&mut session)?;
```

That shape deliberately refuses transparent persistence. Reading requires `refresh(&mut session)`. Writing requires `save(&mut session)`. Normal Rust field access is just Rust field access.

Materialization profiles make the remote read policy visible:

```
use gemstone_rs::{ArrayMaterialization, DictionaryKeyPolicy, MaterializationProfile};

let profile = MaterializationProfile::deep(4)
    .with_dictionary_key_policy(DictionaryKeyPolicy::Preserve)
    .with_array_materialization(ArrayMaterialization::Materialize);
```

Use `MaterializationProfile::shallow()` when you only want an opaque object reference, `DictionaryKeyPolicy::StringOnly` when a JSON-like shape must reject symbol-keyed dictionaries, and `ArrayMaterialization::Reject` when arrays should not appear on a mapping path.

Connector-style config can now record the Smalltalk selector side of a mapping:

```
mapped = Booking
class = UserGlobals:OkzBooking
```

```
field = Booking.status | selector=status | return=Symbol
field = Booking.customer | selector=customer | return=Mapped<Customer>
```

The repo includes a fuller sample at `examples/codegen/connector-mapping.codegen`. It is useful when a team wants to review the Smalltalk selector contract, return shapes, dictionary fallback keys, and relationship mapping before generating or editing wrappers:

```
gemstone-rs codegen explain examples/codegen/connector-mapping.codegen
```

That is the Rust answer to the GemStone-Pharo-Bridge connector idea: keep the mapping reviewable, keep the OOP visible, and let tools generate wrappers or reports from the config without pretending the network is local memory.

For one-off scripts, the typed BridgeRoot helpers avoid manual conversion:

```
bridge_root.put_string("BookingStatus", "ready")?;
bridge_root.put_smallint("BookingAmount", draft.amount)?;
bridge_root.put_bool("BookingApproved", true)?;
bridge_root.put_vec("BookingTags", &["priority".to_string(), "demo".to_string()])?;
bridge_root.put_optional("BookingNote", &Some("front desk".to_string()))?;
bridge_root.put_map("BookingLabels", &draft.labels)?;

assert_eq!(bridge_root.get_string("BookingStatus")?, "ready");
assert_eq!(bridge_root.get_smallint("BookingAmount")?, draft.amount);
assert!(bridge_root.get_bool("BookingApproved")?);
let tags: Vec<String> = bridge_root.get_vec("BookingTags")?;
let note: Option<String> = bridge_root.get_optional("BookingNote")?;
let labels: BTreeMap<String, String> = bridge_root.get_map("BookingLabels")?;
```

When the shape is still exploratory, read the same payload back as a dynamic `BridgeValue` tree instead of committing to a struct:

```
let dynamic = bridge_root.get_bridge_value("BookingDraft")?;
println!("{dynamic:?}");
```

That reads nested dictionaries, arrays, strings, symbols, booleans, small integers, and `nil` into plain Rust data. If a value is outside that supported BridgeRoot shape, or the read hits the configured depth limit, it stays as an explicit `BridgeValue::Oop`. This is useful in the explorer and VS Code webview because users can inspect a live payload first, then turn the stable parts into `BridgeMapped` structs or codegen config.

The terminal workflow is:

```
gemstone-rs bridge value BookingDraft --depth 4
gemstone-rs bridge shape BookingDraft --depth 4
gemstone-rs bridge mapping-preview BookingDraft --mapped BookingDraft --depth 4
```

`shape` prints relationship paths and counts before you generate anything:

```
BridgeValue shape
  total_nodes: 11
  dictionary_nodes: 3
  array_nodes: 1
  unique_oops: 1
  repeated_oop_refs: 1
relationships:
  value.customer.#name    kind=string    key_type=Symbol
  value.items[1].sku      kind=string    key_type=String
  value.items[2]          kind=oop       identity_id=1
  value.items[3]          kind=oop       identity_id=1 repeated_identity=true
repeated identities:
  identity_id=1 oop=1234 paths=value.items[2], value.items[3]
```

`mapping-preview` is the bridge between exploration and codegen. It reads the same live `BridgeValue` tree and emits reviewable config:

```
mapped = BookingDraft | doc=Inferred from a live BridgeRoot value.
field = BookingDraft.customer | type=Mapped<BookingDraftCustomer> | key=customer |
key_type=String
field = BookingDraft.items | type=Vec<Mapped<BookingDraftItem>> | key=items |
key_type=String
field = BookingDraft.note | type=Option<Oop> | key=note | key_type=String |
doc=Observed nil; choose a narrower Option<T> before committing generated code.
```

This keeps the Rust model honest: it helps discover a shape, but the developer still reviews field names, symbol/string key policy, optional fields, opaque OOPs, and repeated object references before generating code.

When a Smalltalk-facing dictionary should use symbols, use the matching key-policy variants for the containing dictionary lookup:

```
bridge_root.put_map_with_key_type("BookingLabels", BridgeKeyType::Symbol,
&draft.labels)?;
let labels: BTreeMap<String, String> =
    bridge_root.get_map_with_key_type("BookingLabels", BridgeKeyType::Symbol)?;
```

That stores the `BookingLabels` entry under a symbol key in BridgeRoot. The map entries themselves remain string-keyed; use `BridgeValue::keyed_dictionary` for symbol-keyed entries inside the stored dictionary value.

Generate a mapping proposal from a live stone:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

Then preview, diff, check, and generate as usual:

```
gemstone-rs codegen preview gemstone-rs.codegen
gemstone-rs codegen diff gemstone-rs.codegen
gemstone-rs codegen check gemstone-rs.codegen
gemstone-rs codegen generate gemstone-rs.codegen
```

The local explorer and VS Code extension expose this workflow too:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/shape?key=BookingDraft&depth=4'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-preview?
key=BookingDraft&mapped=BookingDraft&depth=4'
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
```

The explorer stays read-only unless started with `--allow-write`, so these BridgeRoot write endpoints are useful smoke tests without becoming accidental public write APIs. BridgeRoot reads now return a structured payload with the root name, key, key type, OOP, class OOP, `printString`, and a nested `BridgeValue` tree, so the browser and VS Code webview can render object values as inspection cards instead of raw JSON.

In VS Code, use:

- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: List BridgeRoot Keys`
- `GemStone RS: Put BridgeRoot String`
- `GemStone RS: Put BridgeRoot Symbol`
- `GemStone RS: Put BridgeRoot SmallInt`
- `GemStone RS: Put BridgeRoot Bool`
- `GemStone RS: Remove BridgeRoot Key`
- `GemStone RS: Run Generated Mapping Example`

This is still explicit object mapping, not transparent persistence. That is the right tradeoff for the first Rust layer: Rust callers can review every field, choose string or symbol keys, keep OOPs visible, and still avoid repetitive dictionary boilerplate.

---

# Local Explorer

Run:

```
gemstone-rs-explorer --port 8787
```

For write-enabled local sessions, add a local token:

```
export GEMSTONE_RS_EXPLORER_TOKEN='replace-with-a-local-random-token'
gemstone-rs-explorer --port 8787 --auth-token-env GEMSTONE_RS_EXPLORER_TOKEN
```

Then open `http://127.0.0.1:8787/?token=replace-with-a-local-random-token`. The VS Code workbench can generate and store that token with `GemStone RS: Generate Explorer Auth Token`.

Open:

```
http://127.0.0.1:8787/
```

Useful endpoints:

```
/api/status
/api/browse/dictionaries
/api/browse/classes?dictionary=UserGlobals
/api/browse/methods?class=Object&protocol=--%20all%20--
/api/browse/source?class=Object&selector=printString
/api/codegen/preview?config=examples/codegen/gemstone-rs.codegen
/api/codegen/preview-profile?profile=default&profile_file=examples/codegen/gemstone-
rs.codegen-profiles.json
/api/codegen/diff?config=examples/codegen/gemstone-rs.codegen
/api/codegen/diff-profile?profile=default&profile_file=examples/codegen/gemstone-
rs.codegen-profiles.json
/api/codegen/check-profile?profile=default&profile_file=examples/codegen/gemstone-
rs.codegen-profiles.json
/api/codegen/profiles?profile_file=examples/codegen/gemstone-rs.codegen-profiles.json
/api/codegen/profiles/check?profile_file=examples/codegen/gemstone-rs.codegen-
profiles.json
```

The explorer home page is now a usable UI rather than just a list of links. It lets you browse dictionaries/classes/protocols/methods/source, inspect BridgeRoot, list keys, run codegen checks, and exercise write-gated BridgeRoot edits from a browser.

The codegen workflow now has project profiles. A profile captures the codegen root, config path, mapped Rust type, and GemStone class. Local profiles stay in browser storage, while project profiles can live in a checked-in file such as:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

That means a team can keep named workflows like `default`, `object-wrapper`, and `bridge-mapping` beside the codegen config. The explorer can import/export profile JSON, show which imported profiles are new, replaced, or unchanged, and save project profile files only when started with `--allow-write`. Profile-aware preview, diff, check, explain, and generate endpoints let the browser, VS Code, and CI all resolve the same named project profile before operating on generated wrappers. A project-level profile check endpoint summarizes all profiles at once, including ok, stale, and error counts, so release tooling can fail before generated wrappers drift. In the browser, that report renders as a status table with direct Preview, Diff, Check, and Generate buttons for each profile. The explorer can also read the current generated output file directly through `/api/codegen/output` or `/api/codegen/output-profile`, which is useful when a reviewer wants to compare the committed wrapper with a preview or diff without regenerating anything.

The same schema is available from the CLI, so profile files can be checked in CI:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen preview-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen diff-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen explain-profile --json default gemstone-rs.codegen-profiles.json
```

Write endpoints are deliberately constrained. Relative `config=` and `profile_file=` writes stay under the configured codegen root, `..` traversal in `config=`, `profile_file=`, or `root=` is rejected after URL decoding, and absolute write paths require an explicit `--allow-absolute-write-paths` opt-in. Project profile files are schema-validated before writing, including required unique names and string-valued `config`, `root`, `mapped`, and `className` fields.

The explorer binds to loopback by default, starts read-only, and requires explicit flags for eval and write operations.

---

# VS Code Workbench

The VS Code extension adds a GemStone RS sidebar:

- dictionaries

- classes
- protocols
- methods
- method source
- BridgeRoot key listing
- BridgeRoot value inspection
- BridgeRoot put/remove smoke actions
- codegen preview/diff/check/generate
- generated output file opening
- codegen config and project profile file opening
- profile-driven codegen preview/diff/check/generate
- explorer launch
- embedded explorer webview with live browsing and source preview

For a source checkout:

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

The embedded webview now gives the explorer a more IDE-like loop: it can browse live dictionaries, classes, protocols, methods, and source from the side inspector; open method source in a VS Code editor; open configured codegen and profile files; preview or read generated wrappers in an editable pane; open the current generated text as a VS Code draft; save edited generated output back to the configured file after a confirmation prompt; and warn with Launch Explorer, Open Browser, and Copy URL fallbacks when the loopback explorer is not running. The extension stays thin. The Rust CLI remains the contract, which keeps the tooling testable outside VS Code.

For web services, keep GemStone calls on a blocking worker and treat `Session` as thread-local. The repository now includes a standard-library HTTP service example with `/`, `/health/local`, and `/health/gemstone` routes, a checked Axum service under `examples/axum-service`, and a checked Actix service under `examples/actix-service`. The shared `gemstone_rs::web` module now builds the JSON health responses, while `gemstone-rs-axum` and `gemstone-rs-actix` package the framework route handlers. That keeps the core crate dependency-light while proving the async web-service shape. The framework services can start before credentials are configured: `/` and `/health/local` keep responding, while `/health/gemstone` returns a clear `503` JSON error until the GemStone pool is available. A route smoke script checks that contract for both frameworks, including diagnostic and request-trace headers that identify the adapter, route, request id, method, and path that produced each response. The adapters also emit `x-gemstone-rs-request-lifecycle: received,handled` and `x-gemstone-rs-request-duration-us`, giving proxy logs and tests a small framework-neutral lifecycle signal. The checked Axum and Actix services now also install a tiny framework middleware layer and expose `x-gemstone-rs-example-middleware`, `x-gemstone-rs-service`, `x-gemstone-rs-service-version`, `cache-control: no-store`, and `x-content-type-options: nosniff`, so the smoke test

proves packaged GemStone routes still compose with normal application middleware and a small production-style cache/security policy. Run `python3 scripts/framework_route_smoke.py --live` or set `GS_RUN_LIVE_RUST=1` in a credentialed environment when that same smoke test must require `/health/gemstone` to reach the stone and return `{"result":7}`. The newer `SessionWorker` API gives these services a reusable dedicated-thread session lane when opening a session per health request is not enough:

```
use gemstone_rs::{Config, SessionWorker, Value};

let worker = SessionWorker::start(Config::from_env()?)?;
assert_eq!(worker.eval("3 + 4")?, Value::SmallInt(7));
worker.shutdown()?;
```

For real services, `SessionWorkerPool` keeps the same safety rule while spreading calls across a bounded set of GemStone sessions:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};

let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval("3 + 4")?, Value::SmallInt(7));
pool.shutdown()?;
```

That pool now exposes awaitable calls for async Rust services:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};

let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval_async("3 + 4").await?, Value::SmallInt(7));
let printed = pool
    .perform_oop_async(gemstone_rs::Oop::from_smallint(7), "printString", &[])
    .await?;
assert_eq!(pool.fetch_string_async(printed).await?, "7");
pool.shutdown()?;
```

The important safety property does not change: the async task awaits a future, but the GemStone `Session` still lives on the dedicated worker thread. The Axum and Actix adapters now use this async health path directly.

The newest workbench setup check uses the same CLI `gemstone-rs doctor` report, and the CLI also has `doctor --json`, so terminal diagnostics, VS Code diagnostics, and release automation can all agree.

---

# What to Run First

From the repository root:

```
cargo run -p gemstone-rs --example quickstart
cargo run -p gemstone-rs --example browser
cargo run -p gemstone-rs --example live_smoke_cookbook
cargo run -p gemstone-rs --example oop_values
cargo run -p gemstone-rs --example transactions
cargo run -p gemstone-rs --example session_worker
cargo run -p gemstone-rs --example session_worker_pool
cargo run -p gemstone-rs --example codegen_workflow
cargo run -p gemstone-rs --example http_service -- --routes
cargo run -p gemstone-rs-cli -- compare gemstone-py --gaps
cargo run -p gemstone-rs-cli -- examples scaffold quickstart /tmp/gemstone-rs-
quickstart --force
cargo run -p gemstone-rs-cli -- examples scaffold codegen_workflow /tmp/gemstone-rs-
codegen-workflow --force
cargo run -p gemstone-rs-cli -- examples scaffold profile_codegen_workflow /tmp/
gemstone-rs-profile-codegen --force
cargo run -p gemstone-rs-cli -- examples scaffold generated_wrapper_app /tmp/
gemstone-rs-generated-wrapper --force
cargo run -p gemstone-rs-cli -- examples scaffold session_worker_pool /tmp/gemstone-
rs-worker-pool --force
cargo run -p gemstone-rs-cli -- examples scaffold axum_service /tmp/gemstone-rs-axum-
service --force
cargo run -p gemstone-rs-cli -- examples scaffold actix_service /tmp/gemstone-rs-
actix-service --force
```

For CI and release confidence:

```
make verify
DRY_RUN=1 scripts/release_all.sh 0.2.2
scripts/publish_verify.sh 0.2.2
```

The release wrapper is now the single path for local and GitHub Actions releases: it verifies Rust, codegen, docs PDFs, and VS Code packaging; builds checksums; optionally publishes crates and the VSIX; and can verify crates.io, Marketplace, and GitHub Release assets after publishing.

---

# Where gemstone-rs Fits

Use `gemstone-rs` when you want Rust services, CLIs, workers, explorers, or developer tools to talk to GemStone/S directly. Use `gemstone-py` when your application is Python-native. The shared design direction is clear: keep the low-level bridge small, make the session model explicit, and build higher-level tooling on top of the same stable API.

The native direction is now concrete: `gemstone-py-native` has an additive PyO3 bridge over the Rust core. Rust owns the shared GCI/session bridge, and Python keeps the ergonomic Python API on top.

That direction now has a concrete Rust-side contract. `gemstone_rs::py_native` exposes plain Rust config, value, error, capability, and session wrappers that a PyO3 crate can wrap without duplicating GCI loading or session behavior:

```
use gemstone_rs::py_native::{PyNativeSession, PyNativeValue};

let mut session = PyNativeSession::login_from_env()?;
assert_eq!(session.eval("3 + 4")?, PyNativeValue::SmallInt(7));
let printed = session.perform_values(PyNativeValue::SmallInt(7), "printString",
&[])?;
```

The live smoke example is intentionally small:

```
cargo run -p gemstone-rs-cli -- py-native samples --json
cargo run -p gemstone-rs-cli -- py-native migration --json
cargo run -p gemstone-rs-cli -- py-native compatibility --json
cargo run -p gemstone-rs-cli -- py-native conformance --json
cargo run -p gemstone-rs-cli -- py-native handoff --json
cargo run -p gemstone-rs-cli -- py-native publish-receipt --json
cargo run -p gemstone-rs-cli -- py-native check-all --json
cargo run -p gemstone-rs --example python_native_adapter -- --dry-run
cargo run -p gemstone-rs --example python_native_adapter
```

The samples JSON is useful for wrapper tests because it contains concrete payloads for `nil`, booleans, small integers, characters, strings, symbols, OOPs, and structured errors such as `missingConfig`, `illegalOop`, `unexpectedType`, and path-aware mapping failures.

The migration JSON is useful for release work because it names the shared-core steps and their current status: keep the downstream `RustCoreSession` bridge green, preserve existing Python return behavior, run the native/live smoke suite through the Rust-backed path, and verify published wheels from TestPyPI/PyPI.

The generated PyO3 starter also exposes that report as `gemstone_py_native.migration_json()`, and the real downstream bridge exposes the same `rust_core_*` report shape from `gemstone_py_native._gci`. Its `NativeSession` now also exposes the direct Rust adapter operations for eval, execute, resolve, value-to-OOP conversion, perform, strings, symbols, globals, export-set retention, commit, and abort, while leaving Pythonic return conversion in the Python package layer. `eval_json` and `perform_json` return the stable `PyNativeValue` JSON shape so the Python package can decode values without duplicating native classification rules.

The starter now includes that package layer as executable guidance: `python/gemstone_py_native_compat.py`. It defines `OopHandle` and `NativeCompatibilitySession`, so object identity returned by the direct native module is wrapped before normal Python package

code sees it. That keeps the Rust/PyO3 boundary simple and still lets `gemstone-py-native` preserve its existing Python return behavior by default, with typed helpers kept as explicit opt-in calls. Value-returning helpers become Python dictionaries and object identity becomes `OopHandle`, which keeps the backward-compatible policy visible in code instead of hidden inside the extension module.

There is now a machine-readable compatibility report too: `gemstone-rs py-native compatibility --json`. It lists each generated Python shim method, the underlying `NativeSession` method, the native return type, and the Python return type. That turns a vague "keep Python compatible" requirement into a fixture-backed checklist that wrapper CI can diff before the real `gemstone-py-native` package is changed.

The next layer is a conformance report: `gemstone-rs py-native conformance --json`. It names the extension module functions, raw `NativeSession` methods, compatibility shim methods, checked-in fixtures, and generated scaffold files that the real `gemstone-py-native` wrapper should expose. That makes the shared-core handoff testable from Rust CI, Python CI, and the VS Code workbench without requiring a live stone.

The final Rust-side bundle is `gemstone-rs py-native handoff --json`. It collects the capabilities, samples, smoke, migration, compatibility, and conformance artifacts into one manifest and adds the required acceptance checks: scaffold compile, fixture freshness, preserved Python return policy, live Rust-core native smoke, and published wheel verification from TestPyPI and PyPI. The generated PyO3 starter exposes the same manifest as `gemstone_py_native.handoff_json()`, and the real native package exposes the Rust-core report functions under `gemstone_py_native._gci`, so release gating stays close to the package that publishes the wheels.

There is also a publish receipt: `gemstone-rs py-native publish-receipt --json`. It records the exact TestPyPI and PyPI workflow run ids, install commands, verified timestamps, and package checks for the Rust-backed `gemstone-py-native` wheel release. That makes the release evidence durable and reviewable in source control instead of buried in CI logs.

The release gate is `gemstone-rs py-native check-all`. It validates every checked-in Rust-side fixture in one pass: capabilities, samples, smoke, compatibility, conformance, handoff, and publish receipt. The JSON output is intentionally boring. That is the point. It gives the eventual `gemstone-py-native` workflow one stable command to run before wheels are published.

The shared-core release path is now closed for this phase. Local live GemStone smoke through the Rust-backed native path passed, the Rust-backed `gemstone-py-native` wheels were published, and TestPyPI/PyPI install verification passed for those wheels.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.